1. Depth-First Search (DFS) Algorithm:

    a. Explain how the DFS algorithm works, and describe it using pseudocode or an algorithm.

Depth-First Search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root (or an arbitrary node) and explores as far as possible along each branch before backtracking. It uses a stack (often implicitly through recursion) to remember the nodes to visit next.

    b. Implement the 8-puzzle problem using the DFS algorithm in Python.

```python
import collections
import time

class EightPuzzleDFS:
    """
    Implements the 8-puzzle problem solver using Depth-First Search (DFS).
    """
    def __init__(self, start_state, goal_state):
        # Convert list of 9 elements into a tuple for hashability (for set/dict keys)
        self.start_state = tuple(start_state)
        self.goal_state = tuple(goal_state)
        self.visited = set()
        self.max_depth = 20  # Limit the depth to prevent excessive runtime/stack overflow
        self.solution_path = []

    def get_blank_position(self, state):
        """Returns the index (0-8) of the blank tile (0)."""
        return state.index(0)

    def get_possible_moves(self, state):
        """Generates all valid successor states by moving the blank tile."""
        i = self.get_blank_position(state)
        r, c = i // 3, i % 3  # Current row and column

        # Possible (row_change, col_change) moves: (Up, Down, Left, Right)
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        successors = []

        for dr, dc in moves:
            nr, nc = r + dr, c + dc  # New row and column

            # Check if the new position is within the 3x3 grid
            if 0 <= nr < 3 and 0 <= nc < 3:
                ni = nr * 3 + nc  # New index

                # Create the new state tuple
                new_state_list = list(state)
                # Swap the blank tile (0) with the tile at the new position
                new_state_list[i], new_state_list[ni] = new_state_list[ni], new_state_list[i]
                successors.append(tuple(new_state_list))

        return successors

    def solve(self):
        """Initiates the DFS search."""
        print(f"Start State: {self.format_state(self.start_state)}")
        print(f"Goal State:  {self.format_state(self.goal_state)}")
```

```
60
61          while frontier:
62              current_state, path, depth = frontier.pop() # LIFO behavior for DFS
63
64              if current_state == self.goal_state:
65                  self.solution_path = path
66                  end_time = time.time()
67                  print("\n*** Solution Found! ***")
68                  print(f"Search Time: {end_time - start_time:.4f} seconds")
69                  print(f"Path Length: {len(path) - 1} moves")
70                  print("Solution Path:")
71                  for step, state in enumerate(path):
72                      print(f"Step {step}:")
73                      print(self.format_state(state))
74                  return True
75
76              if depth < self.max_depth:
77                  # Get successors and push them onto the stack (LIFO)
78                  # We reverse the successors before adding them to ensure standard exploration order
79                  for successor in reversed(self.get_possible_moves(current_state)):
80                      if successor not in self.visited:
81                          self.visited.add(successor)
82                          new_path = path + [successor]
83                          frontier.append((successor, new_path, depth + 1))
84
85          print("\n*** Search Failed ***")
86          print(f"No solution found within the maximum depth of {self.max_depth}.")
87          return False
88
89      def format_state(self, state):
90          """Formats the tuple state into a readable 3x3 grid."""
91          s = ""
92          for i in range(0, 9, 3):
93              s += f"| {state[i]} {state[i+1]} {state[i+2]} |\n"
94          # Replace 0 with a blank space for better visualization
95          return s.replace('0', ' ').strip()
96
97  # --- Example Usage ---
98  # Solvable (4 moves from start to goal)
99  start = [1, 2, 3, 4, 5, 6, 7, 8, 0] # Goal state moved 0 right to 8
100 goal = [1, 2, 3, 4, 5, 6, 7, 0, 8] # Goal state
101
102 # A slightly more complex but solvable example
103 start = [1, 2, 3, 0, 4, 6, 7, 5, 8]
104 goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
105
106 # Initialize and solve
107 puzzle_solver = EightPuzzleDFS(start, goal)
108 puzzle_solver.solve()
```

c. Include screenshots of your code and its output. Please write your name as a comment in the code.

```python
# My Name: Rijin
import collections
import time

class EightPuzzleDFS:
    """
    Implements the 8-puzzle problem solver using Depth-First Search (DFS).
    """
    def __init__(self, start_state, goal_state):
        # Convert list of 9 elements into a tuple for hashability (for set/dict keys)
        self.start_state = tuple(start_state)
        self.goal_state = tuple(goal_state)
        self.visited = set()
        self.max_depth = 20  # Limit the depth to prevent excessive runtime/stack overflow
        self.solution_path = []

    def get_blank_position(self, state):
        """Returns the index (0-8) of the blank tile (0)."""
        return state.index(0)

    def get_possible_moves(self, state):
        """Generates all valid successor states by moving the blank tile."""
        i = self.get_blank_position(state)
        r, c = i // 3, i % 3  # Current row and column

        # Possible (row_change, col_change) moves: (Up, Down, Left, Right)
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        successors = []

        for dr, dc in moves:
            nr, nc = r + dr, c + dc  # New row and column

            # Check if the new position is within the 3x3 grid
            if 0 <= nr < 3 and 0 <= nc < 3:
                ni = nr * 3 + nc  # New index
```

```
| 1 2 3 |
| 4   6 |
| 7 5 8 |
Step 2:
| 1 2 3 |
| 4 5 6 |
| 7   8 |
Step 3:
| 1 2 3 |
| 4 5 6 |
| 7 8   |
PS C:\Users\ACER\Desktop\instadraw>
```

d. Discuss why, in some cases, DFS may fail to find a solution for this problem.

For the 8-puzzle problem, DFS may fail to find an optimal solution, or even *any* solution within a reasonable time, due to two main reasons:

a. Non-Optimality (Finding the Long Path): DFS is not guaranteed to find the shortest path. It dives deep into one branch first. If the shortest path is at a shallow depth, but the algorithm chose a very long, irrelevant path first, it will find the long solution and stop, ignoring the optimal one.

b. Infinite Loop/Path Exploration (Failure to Terminate): The state space of the 8-puzzle is finite ($9!/2 = 181,440$ reachable states). However, without a mechanism to track visited states (as implemented in the code above), DFS can get trapped in a loop, repeatedly moving between the same few states on a branch, especially in a graph structure. Even *with* cycle detection, DFS is not complete in a graph with infinite depth paths or an infinite state space. Although the 8-puzzle is finite, DFS's tendency to explore one path to its maximum depth means that if the goal is at a shallow depth, but the algorithm is stuck exploring an extremely long or redundant path, it will effectively fail to find the solution within a practical time limit or the maximum recursive depth/memory limit. The depth limit in the provided code is a common, non-guaranteed way to prevent this practical failure.

2. Breadth-First Search (BFS) State Space Design:

    a. While implementing BFS in Python, did you first construct the entire state space (tree or graph) before starting the search, or did you build it dynamically during the search?

    When implementing BFS in Python, I would build the state space dynamically during the search, rather than constructing the entire graph/tree beforehand.

    Approach and Justification

- Approach: The BFS algorithm uses a queue (First-In, First-Out, FIFO) to manage the frontier of states to be explored.

    1. Start by adding the initial state to the queue.
    2. Maintain a visited set to store all states encountered.
    3. In each step, dequeue the next state.
    4. If it's the goal, stop.
    5. Otherwise, generate its successors (neighbors).
    6. For each *new, unvisited* successor, add it to the visited set and enqueue it.

    b. Explain the approach you took, and justify why you chose that method.

 State Space Size: The 8-puzzle has a relatively small state space (181,440 reachable states). However, for larger problems, such as the 15-puzzle (10^{13} reachable states) or problems with a theoretically infinite state space (like an agent moving in an unbounded grid), constructing the entire state space in memory is impossible or impractical.

 Memory Efficiency: Building the state space dynamically means we only store the nodes in the frontier (queue) and the nodes already visited, which is generally a small fraction of the total space at any given time, saving significant memory.

 Completeness/Relevance: Since BFS guarantees finding the shortest path (it explores layer-by-layer), we are guaranteed to find the goal state the first time we encounter it. Generating irrelevant parts of the search space beforehand would be a waste of computation. We only generate the states *necessary* to connect the start state to the goal state.

3. Informed Search vs. BFS:
    a. What do you understand by the term informed search?

Informed search, also known as heuristic search, is a type of search strategy that uses problem-specific knowledge beyond the definition of the state space. This knowledge, called a heuristic function , estimates the cost from the current state to the goal state. This allows the search algorithm to explore the most promising states first, significantly improving efficiency compared to uninformed methods.

    b. How does informed search differ from Breadth-First Search (BFS)?

BFS blindly explores neighbors in a level-by-level manner, whereas Informed Search strategically guides the search towards the goal using an educated guess (the heuristic value).

4. Heuristic Values in Informed Search:
    a. Define heuristic values in the context of informed search algorithms.

A heuristic value, denoted AS, in the context of informed (or heuristic) search algorithms is an educated estimate of the cost of the cheapest path from the current state to the goal state.

    b. List a few common heuristic functions and mention their application areas.

1. Manhattan Distance	The sum of the absolute differences in the x and y coordinates of a tile's current position and its goal position. (Also known as "city block" distance).	**Sliding-Tile Puzzles** (e.g., 8-puzzle, 15-puzzle) and **Grid-based Pathfinding** where movement is restricted to cardinal directions (Up, Down, Left, Right).
2. Misplaced Tiles	The count of tiles that are currently not in their correct goal position. (Also known as Hamming Distance for the 8-puzzle).	Simple and computationally cheap heuristic for **Sliding-Tile Puzzles**.
3. Euclidean Distance	The straight-line or "as the crow flies" distance between the current state and the goal state, calculated using the Pythagorean theorem.	**Pathfinding** in continuous spaces or graphs where movement cost is proportional to the geometric distance and is not restricted to a grid.
4. Minimum Spanning Tree (MST) Heuristic	The cost of the minimum spanning tree constructed over the set of unvisited nodes (cities) plus the current node and the goal node.	**Traveling Salesperson Problem (TSP)** and other complex **Routing and Optimization Problems**.

5. A* Algorithm for Binary String Matching:
   a. How can the A* algorithm be applied to solve the problem of binary string matching?
   Example: From the start state 10110101 to the goal state 01100011.

Component	Definition	Example Value
State	The current binary string.	`10110101`
Start State (S_{start})	The initial binary string.	`10110101`
Goal State (S_{goal})	The target binary string.	`01100011`
Action/Operator	The allowed transformation between states, typically defined as **flipping a single bit**.	Flip the 1st bit: `10110101` $\rightarrow$ `00110101`
Cost of Action	The cost of a single operation, usually **1** (uniform cost).	$g(n) = 1$ for each flip.
Evaluation Function	$f(n) = g(n) + h(n)$	Cost so far + Estimated cost to go

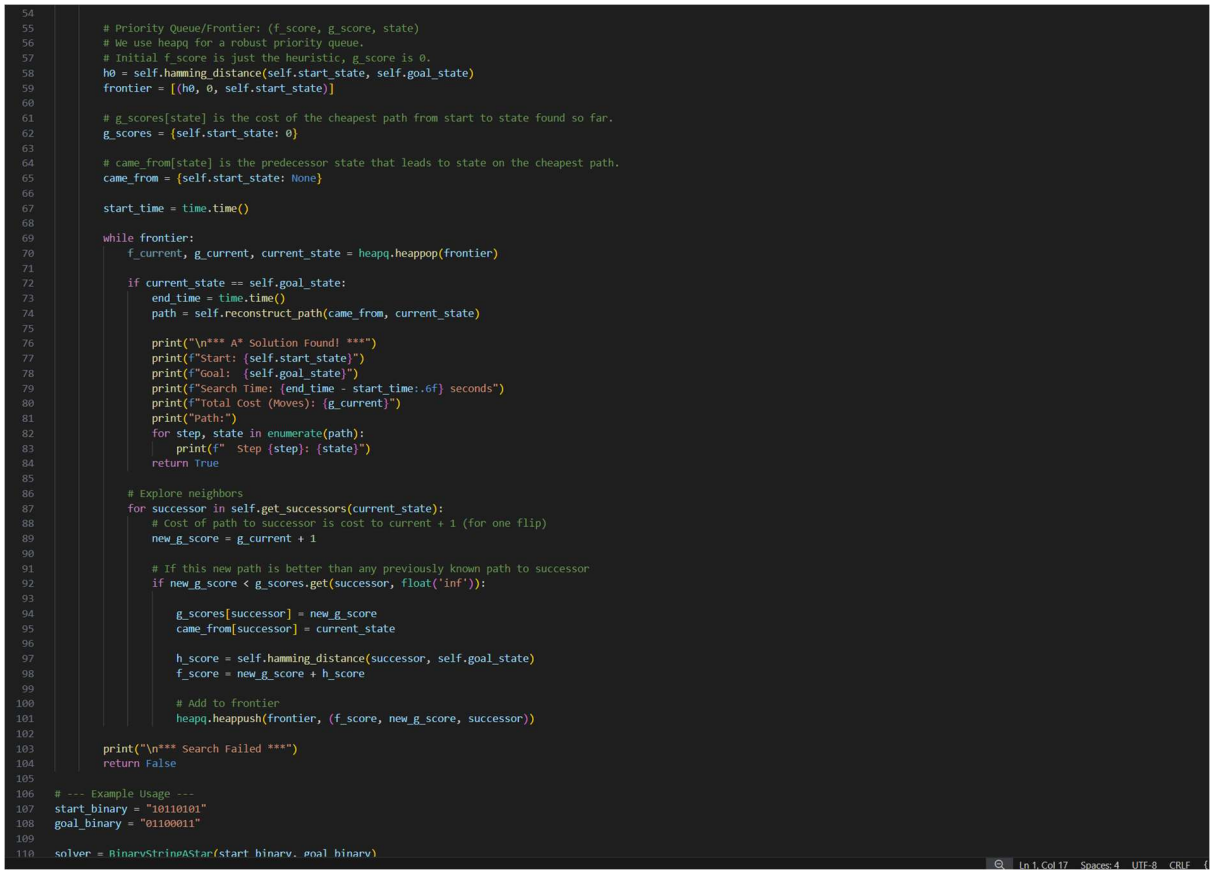
   b. Implement the A* algorithm in Python to solve this problem.

```python
# My Name: Rijin
import heapq
import time

class BinaryStringAStar:
    """
    Solves the binary string matching problem using A* search.
    Action: Flipping a single bit (cost = 1).
    Heuristic: Hamming Distance (Number of differing bits).
    """
    def __init__(self, start, goal):
        self.start_state = start
        self.goal_state = goal

    def hamming_distance(self, s1, s2):
        """
        Computes the heuristic value h(n): the number of differing bits.
        This is the minimum number of flips required to reach s2 from s1.
        """
        distance = 0
        for b1, b2 in zip(s1, s2):
            if b1 != b2:
                distance += 1
        return distance

    def get_successors(self, state):
        """
        Generates all successor states by flipping a single bit.
        """
        successors = []
        state_list = list(state)
        for i in range(len(state_list)):
            # Flip the bit at position i
            original_bit = state_list[i]
            new_bit = '0' if original_bit == '1' else '1'

            # Create the new state string
            new_state_list = state_list[:]
            new_state_list[i] = new_bit
            successors.append("".join(new_state_list))
        return successors

    def reconstruct_path(self, came_from, current):
        """Reconstructs the path from start to current."""
        path = [current]
        while current in came_from:
            current = came_from[current]
            if current: # Check for the None start state
                path.append(current)
        return path[::-1] # Reverse the path to go from start to goal

    def solve(self):
        """Performs the A* search."""

        # Priority Queue/Frontier: (f_score, g_score, state)
        # We use heapq for a robust priority queue.
        # Initial f_score is just the heuristic, g_score is 0.
        h0 = self.hamming_distance(self.start_state, self.goal_state)
        frontier = [(h0, 0, self.start_state)]
```

c. Include screenshots of your code and output, and write your name as a comment in the code.

```
PS C:\Users\ACER\Desktop\instadraw> python -u "c:\Users\ACER\Desktop\instadraw\tempCodeRunnerFile.python"

*** A* Solution Found! ***
Start: 10110101
Goal:  01100011
Search Time: 0.000288 seconds
Total Cost (Moves): 5
Path:
  Step 0: 10110101
  Step 1: 00110101
  Step 2: 00100101
  Step 3: 00100001
  Step 4: 00100011
  Step 5: 01100011
PS C:\Users\ACER\Desktop\instadraw>
```

d.



```
        # Priority Queue/Frontier: (f_score, g_score, state)
        # We use heapq for a robust priority queue.
        # Initial f_score is just the heuristic, g_score is 0.
        h0 = self.hamming_distance(self.start_state, self.goal_state)
        frontier = [(h0, 0, self.start_state)]

        # g_scores[state] is the cost of the cheapest path from start to state found so far.
        g_scores = {self.start_state: 0}

        # came_from[state] is the predecessor state that leads to state on the cheapest path.
        came_from = {self.start_state: None}

        start_time = time.time()

        while frontier:
            f_current, g_current, current_state = heapq.heappop(frontier)

            if current_state == self.goal_state:
                end_time = time.time()
                path = self.reconstruct_path(came_from, current_state)

                print("\n*** A* Solution Found! ***")
                print(f"Start: {self.start_state}")
                print(f"Goal:  {self.goal_state}")
                print(f"Search Time: {end_time - start_time:.6f} seconds")
                print(f"Total Cost (Moves): {g_current}")
                print("Path:")
                for step, state in enumerate(path):
                    print(f"  Step {step}: {state}")
                return True

            # Explore neighbors
            for successor in self.get_successors(current_state):
                # Cost of path to successor is cost to current + 1 (for one flip)
                new_g_score = g_current + 1

                # If this new path is better than any previously known path to successor
                if new_g_score < g_scores.get(successor, float('inf')):

                    g_scores[successor] = new_g_score
                    came_from[successor] = current_state

                    h_score = self.hamming_distance(successor, self.goal_state)
                    f_score = new_g_score + h_score

                    # Add to frontier
                    heapq.heappush(frontier, (f_score, new_g_score, successor))

        print("\n*** Search Failed ***")
        return False

# --- Example Usage ---
start_binary = "10110101"
goal_binary = "01100011"

solver = BinaryStringAStar(start_binary, goal_binary)
```

e. Which heuristic function did you use? Explain how your code computes the heuristic values.

Submission Status